1.

4. Se dă un sistem de memorie, unde dimensiunea memoriei principale este de 32 GiB, adresarea este la nivel de byte, blocul este de 32 cuvinte, iar cuvântul este pe 32 biți. La ambele rânduri, maparea este asociativă pe 4 căi. Dimensiunea de date a cache-ului este de 64 KiB.
a) Desenați schema logică a cache-ului, în maniera prezentată la curs, cu etichetarea lățimii semnalelor.
b) Calculați indexul din cache la care se mapează adresa: $04AAF8130_{hex}$.

a) $MM = 32\ GiB = 2^5 \cdot 2^{30} = 2^{35}\ B \Rightarrow p.a. = 35$

$1\ Block = 32\ words = 2^5\ words \Rightarrow B.O. = 5$

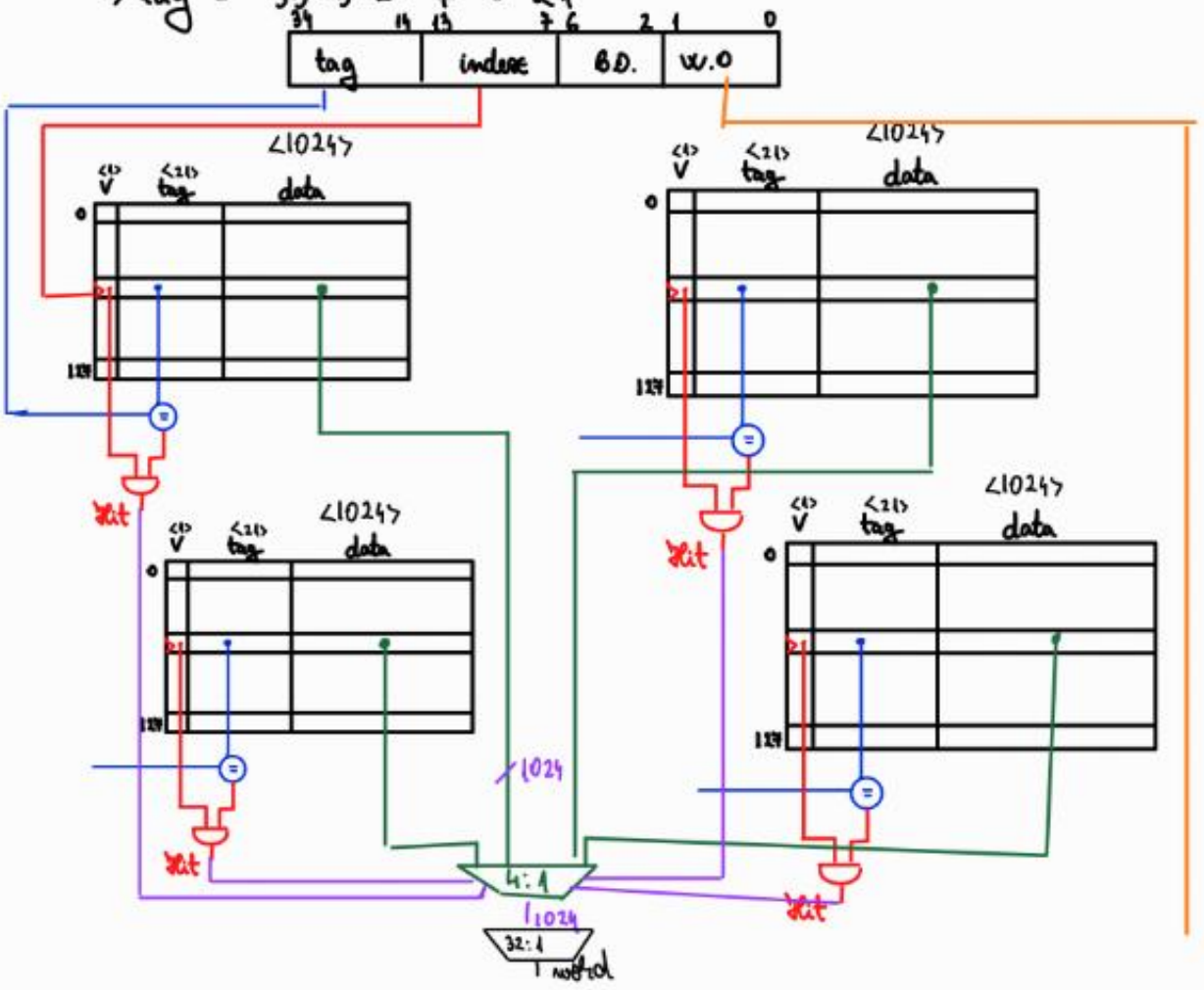
$1\ word = 32\ b = 4\ B = 2^2\ B \Rightarrow W.O. = 2$

4 WAY SA

$data\ cache\ size = 64\ KiB = 2^6 \cdot 2^{10} = 2^{16}\ B$

$\frac{2^{16}}{2^5 \cdot 2^2 \cdot 2^2} = 2^7 \Rightarrow index = 7$

$\Rightarrow tag = 35 - 5 - 2 - 7 = 21$

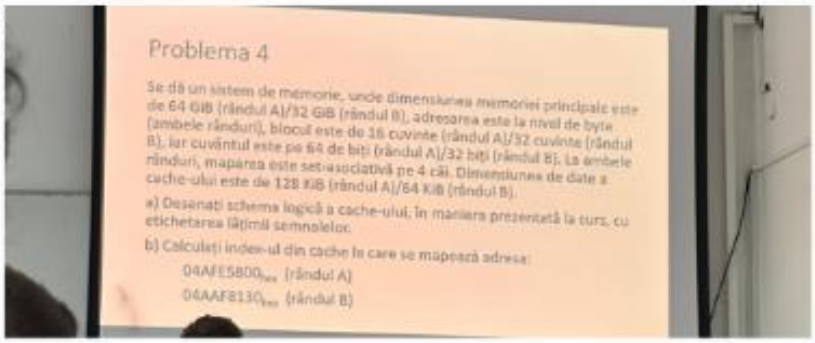




b) 0.4A.A F.8 1.30

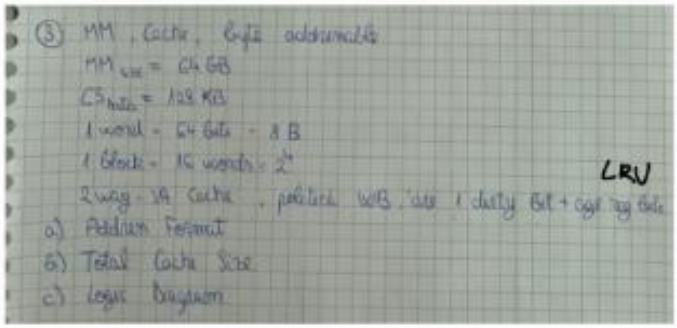
	<21> tag	<7> index	<4> B.O.	<2> W.O	line
0x04AA F.100	0.0001.00		11.00	00	4

Ex 2



la fel ca sus

Ex 3



a) MM = 64 GB = $2^6 \cdot 2^{30} = 2^{36}$ => p.a = 36

CS = 128 KiB = $2^7 \cdot 2^{10} = 2^{17}$

1 word = 64 bits = 8B = 2^3 B => W.O. = 3 b

1 block = 16 words = 2^4 words => B.O. = 4 b

=> $\frac{2^{17}}{2^3 \cdot 2^4 \cdot 2^1} = 2^{10}$ => index = 10

tag = 36 - 10 - 3 - 4 = 19

b) 1b dirty, 1b age (2 way)

total cache size = $2^{10} \cdot 2 \cdot (1b + 1b + 1b + 2^3 \cdot 2^4 \cdot 2^3 + 19b)$ =

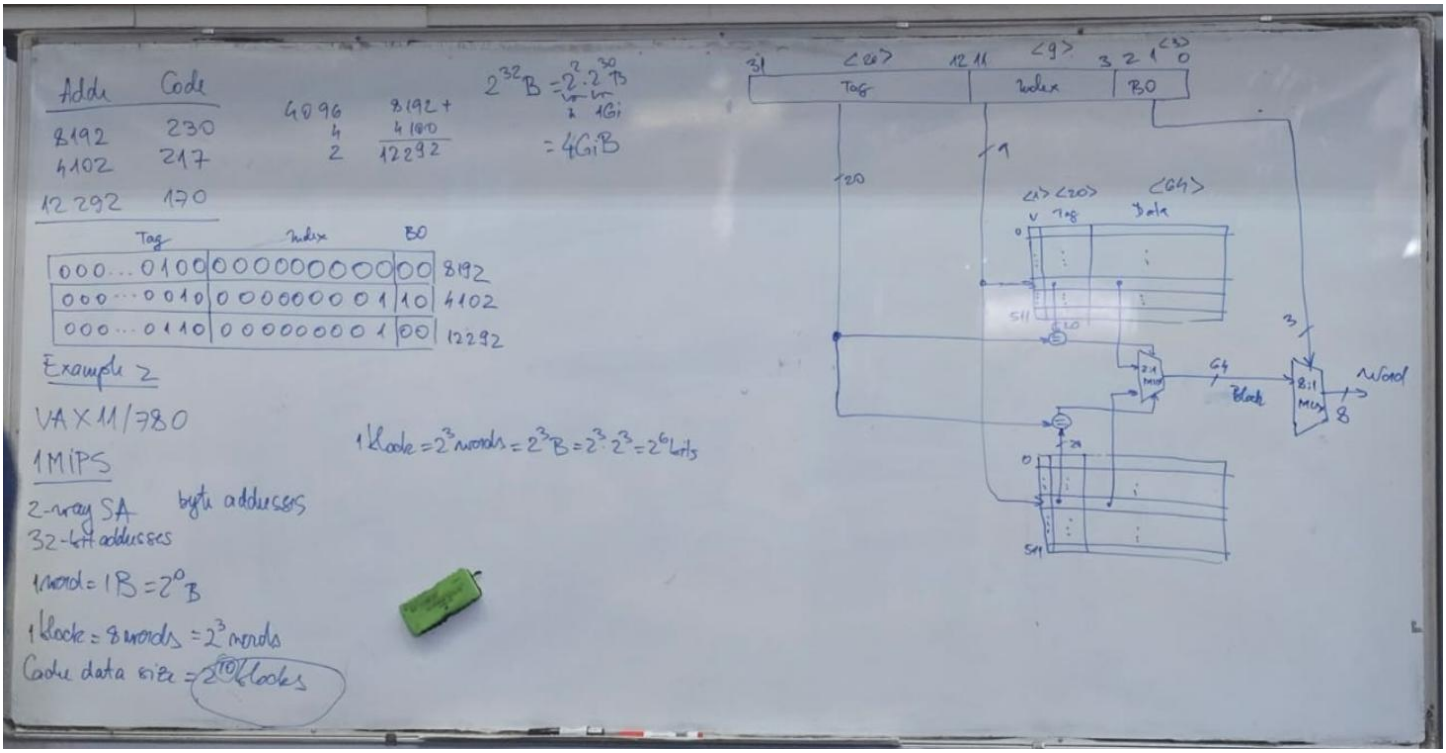
(linii, banks, V, dirty, age, data, tag)

$= 2^{11}(2^{10} + 22b) = 2^{11} \cdot 2^{10} + 2^{11} \cdot 2^2 = 2 \cdot 2^{20} + 2^3 \cdot 2^{10}$ - 2048 KiB + 8 KiB = 2056 KiB

$$X \text{ KiB} = X \cdot 2^{10} \text{ Bytes}$$

$$X \text{ MiB} = X \cdot 2^{20} \text{ Bytes}$$

$$X \text{ GiB} = X \cdot 2^{30} \text{ Bytes}$$



Exc 4.

(3) (2p) Avem un computer load/store cu un sistem cache ce se caracterizează prin următoarele valori empirice atunci când rulăm un anumit program: 10% Data Miss Rate, 8% Instruction Miss Rate și 20% instrucțiuni load/store. În orice moment, 20% din blocuri sunt modificate (i.e., dirty). Programul dezvoltă un CPI de 2 clock cycles dacă cache-ul este perfect și conține din 10 000 de instrucțiuni în cod mașină. Calculatorul are o frecvență a clock-ului de 4.5 GHz. Penalitatea de miss e determinată de faptul că un access la BUS (durează) 10 cicluri de clock, lățimea BUS-ului este de 4 cuvinte, block-ul are 16 cuvinte iar cache-ul implementează o politică Write Back cu Write Allocate în cazul unui miss. Calculați:

(1p) a) AMAT, știind că t_{ac} = 2 ca.

(1p) b) CPU time.

a) Read Miss Penalty = $\underbrace{\frac{16 \text{ words}}{4 \text{ words}} \cdot 0{,}2 \text{ Bus writes}}_{\text{update}}$ + $\underbrace{\frac{16 \text{ words}}{4 \text{ words}} \text{ Bus Reads}}_{\text{allocate}}$ = $4 \cdot 0{,}2 + 4 = 4 \cdot 1{,}2 = 4{,}8$ Bus accesses

=> Miss Penalty = 4,8 · 10 = 48 C.C.

$CCT = \frac{1}{4{,}5 \cdot 10^{-9}} = 0{,}2222\ \mu s$

Misses Per Instruction = Miss Rate · Mem. accesses per instruction (1 + 0,2 → L/S)

$\underset{\text{MPi}}{\text{diff.}} = 1 \times IMR + 0{,}2 \cdot DMR = 0{,}08 + 0{,}2 \cdot 0{,}1 = 0{,}1$

Miss Rate = $\frac{0{,}1}{1{,}2} = 0{,}0833$

$AMAT = \underset{CCT \cdot t_{ac}}{0{,}4444\ \mu s} + \underset{MR \cdot MP \cdot CCT}{0{,}833 \cdot 48CC \cdot 0{,}2222} = 1{,}3328\ \mu s$ (mem Access)

b) $CPU_{TIME} = IC \cdot (CPI_{ideal} + MPi \cdot MP) \cdot CCT =$

$= 10\,000\,(2 + 0{,}1 \cdot 48) \cdot 0{,}2222 = 15{,}1096\ \mu s$

$AMAT = 0{,}4444\ \mu s + 0{,}833 \cdot 48CC \cdot 0{,}2222$

P CCT

Ex 5

(13) Avem un computer L/S cu un sistem cache cu k caracteristici
(12) prin măsurătorile valori empirice atunci când rulăm un anumit program: 9% Data Miss Rate, 8% Instruction Miss Rate și 10% instrucțiuni L/S. În orice moment 10% din blocuri sunt modificate (i.e. dirty). Programul determină un CPI de 3 c.c. dacă cache-ul este perfect și constă din 1000 de instrucțiuni în cod mașină. Calculatorul are o frecvență a clock-ului de 5 GHz. Penalitatea de miss e determinată de faptul că un access la BUS necesită 8 cicluri de clock, lățimea BUS-ului este de 2 cuvinte, blocul are 32 de cuvinte iar cache-ul implementează o politică Write Through cu Write No Allocate în cazul unui miss.
10% ~~dintre~~ din acesse sunt scrieri, restul sunt citiri.

WT + WNA

BMR = 9% = 0,09

IMR = 8% = 0,08

L/S -> 10% = 0,1

10% = 0,1 dirty

CPI_{ideal} = 3 CC

IC = 1000

CR = 5 GHz -> $CCT = \frac{1}{5 \cdot 10^9} = 0,2$ ns

8 CC -> 1 Bus acc.

BUS width = 2 words

Block size = 32 words

10% scrieri

tac = 1 cc

$$RMP_{WT} = \frac{\text{Block size}}{\text{BUS width}} = \frac{32}{2} = 16 \text{ Bus Reads}$$

$WMP_{WT} = 1$

$$\text{Miss Penalty} = (0,1 \cdot 1 + 0,9 \cdot 16)\, 8 = 14,5 \cdot 8 = 116 \text{ cc.}$$

$$\text{Misses / instr} = MR \cdot \underbrace{\text{Mem. acc / instr}}_{1 + 0,1} =$$

$$= 1 \cdot 0,08 + 0,1 \cdot 0,09$$

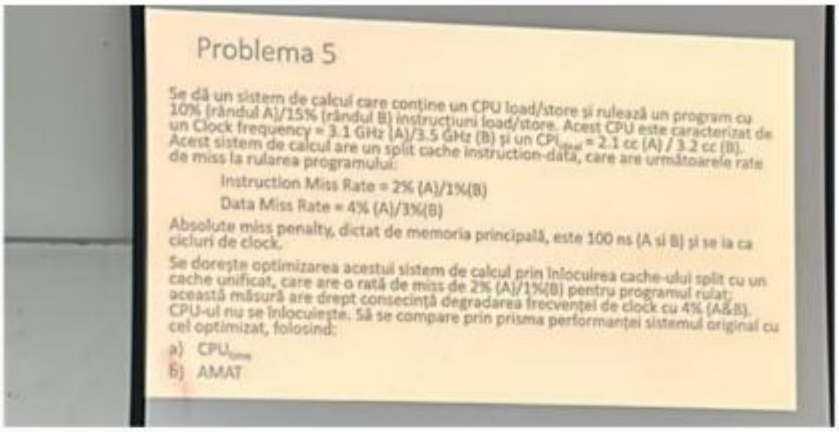
$$\Rightarrow MR = \frac{0,089}{1,1} = 0,081$$

$$AMAT = \underbrace{(1 + MR \cdot MP)}_{\to CC} \cdot CCT = (1 + 0,081 \cdot 116) \cdot 0,2 = 2,079 \text{ ns}$$

b) $$CPU_{time} = ic \cdot (\underbrace{CPI_{ideal} + MR \cdot MP \cdot \text{Mem acc/instr}}_{\to CC}) \cdot CCT =$$

$$= 1000(3 + 0,081 \cdot 116 \cdot 1,1) \cdot 0,2 = 2667 \text{ ns}$$

6.



Problema 5

Se dă un sistem de calcul care conține un CPU load/store și rulează un program cu 10% (rândul A)/15% (rândul B) instrucțiuni load/store. Acest CPU este caracterizat de un Clock frequency = 3.1 GHz (A)/3.5 GHz (B) și un CPI_{ideal} = 2.1 cc (A) / 3.2 cc (B). Acest sistem de calcul are un split cache instruction-data, care are următoarele rate de miss la rularea programului:

Instruction Miss Rate = 2% (A)/1%(B)

Data Miss Rate = 4% (A)/3%(B)

Absolute miss penalty, dictat de memoria principală, este 100 ns (A și B) și se ia ca cicluri de clock.

Se dorește optimizarea acestui sistem de calcul prin înlocuirea cache-ului split cu un cache unificat, care are o rată de miss de 2% (A)/1%(B) pentru programul rulat; această măsură are drept consecință degradarea frecvenței de clock cu 4% (A&B). CPU-ul nu se înlocuiește. Să se compare prin prisma performanței sistemul original cu cel optimizat, folosind:

a) CPU_{time}

b) AMAT

A.

$IC_{LIS} = 10\% = 0,1$

$CR = 3,1\ GHz \Rightarrow CCT = \frac{1}{3,1\ GHz} = 0,32\ ns$

$CPI_{ideal} = 2,1\ cc$

$iMR = 2\% = 0,02$

$dMR = 4\% = 0,04$

$MP = 100\ ns = \lceil \frac{100}{0,32} \rceil = 313\ cc.$

$MR_{new} = 2\% = 0,02$

$\Rightarrow CR_{new} = \frac{96}{100} \cdot CR = 0,96 \cdot 3,1 = 2,98\ GHz$

$\Rightarrow CCT_{new} = \frac{1}{2,98\ GHz} = 0,34\ ns$

a) $CPU_{time} = IC \cdot (CPI_{ideal} + MR \cdot MP \cdot Mem_{acc}/instr) \cdot CCT$ ($MP \rightarrow cc$; $1 + 0,1 \rightarrow I/S$)

$Misses/instr = MR \cdot Mem\ acc/instr$

$= 1 \cdot 0,02 + 0,1 \cdot 0,04$

$\Rightarrow MR = \frac{0,024}{1,1} = 0,022$

$CPU_{time_1} = IC(2,1 + 0,022 \cdot 313 \cdot 1,1) \cdot 0,32 = IC \cdot 2,876\ ns$

$CPU_{time_2} = IC \cdot (2,1 + 0,02 \cdot MP_2 \cdot 1,1) \cdot 0,34 = IC \cdot 2,921\ ns$

$MP_2 = \lceil \frac{100}{0,34} \rceil = 295\ cc$

$\Rightarrow$ 1 is better

b) $AMAT = (1 + MP \cdot MR) \cdot CCT$

$AMAT_1 = (1 + 313 \cdot 0{,}022) \cdot 0{,}32 \sim 2{,}524$ ms

$AMAT_2 = (1 + 295 \cdot 0{,}02) \cdot 0{,}34 = 2{,}336$ ms

2 ist Better

Var 1

4. Mem virtuală 4TB
Mem fizică 256GB
byte addressable
word = 64 biți = 8 bytes
block = 16 words
page = 16 KiB
TLB = 64 linii, direct mapping
Cache-ul 256 KiB, write back și write allocate
4 way SA, LRU ca reg substituție

a) formatele cuvintelor de adresă fizică și virtuală
b) dim totale ale TLB și cache
c) schema comună TLB și cache

VM = 4TB = $2^2 \cdot 2^{40} = 2^{42}$ => V.M. adr = 42 b

PM = 256 GB = $2^8 \cdot 2^{30} = 2^{38}$ => P.M. adr = 38 b

page = 16 KiB = $2^4 \cdot 2^{10} = 2^{14}$ B => Page Offset (P.O) = 14 b

1 word = 64 b = 8B = 2^3 B => W.O. = 3 b

1 block = 16 words = 2^4 words => B.O. = 4 b

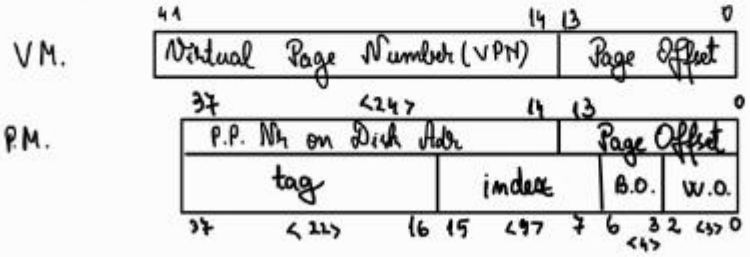
TLB = 64 lines = 2^6 lines => index V.M. = 6 b

cache size = 256 KiB = $2^8 \cdot 2^{10} = 2^{18}$ B => $\frac{2^{18}}{2^3 2^4 \cdot 2^2} = 2^9$ blocks/bank => index = 9 b

WB + WA

4 WAY-SA + LRU

4 way => tag = 38 - 9 - 4 - 3 = 22 b



b) Cache total size = $2^9 \cdot 2^2 \cdot (1\,b + 1\,b + 2\,b + 22\,b + 2^4 \cdot 2^3\,B)$ =

(index, 4way, v, dirty, age, tag)

$= 2^{11} (26\,b + 2^7 B) = 2^{11} \cdot 2^2\,B + 2^{10} \cdot 2^8\,B = 2^{10}(8 + 256)$ =

$= 264$ KiB

(32b : 8 = 4)

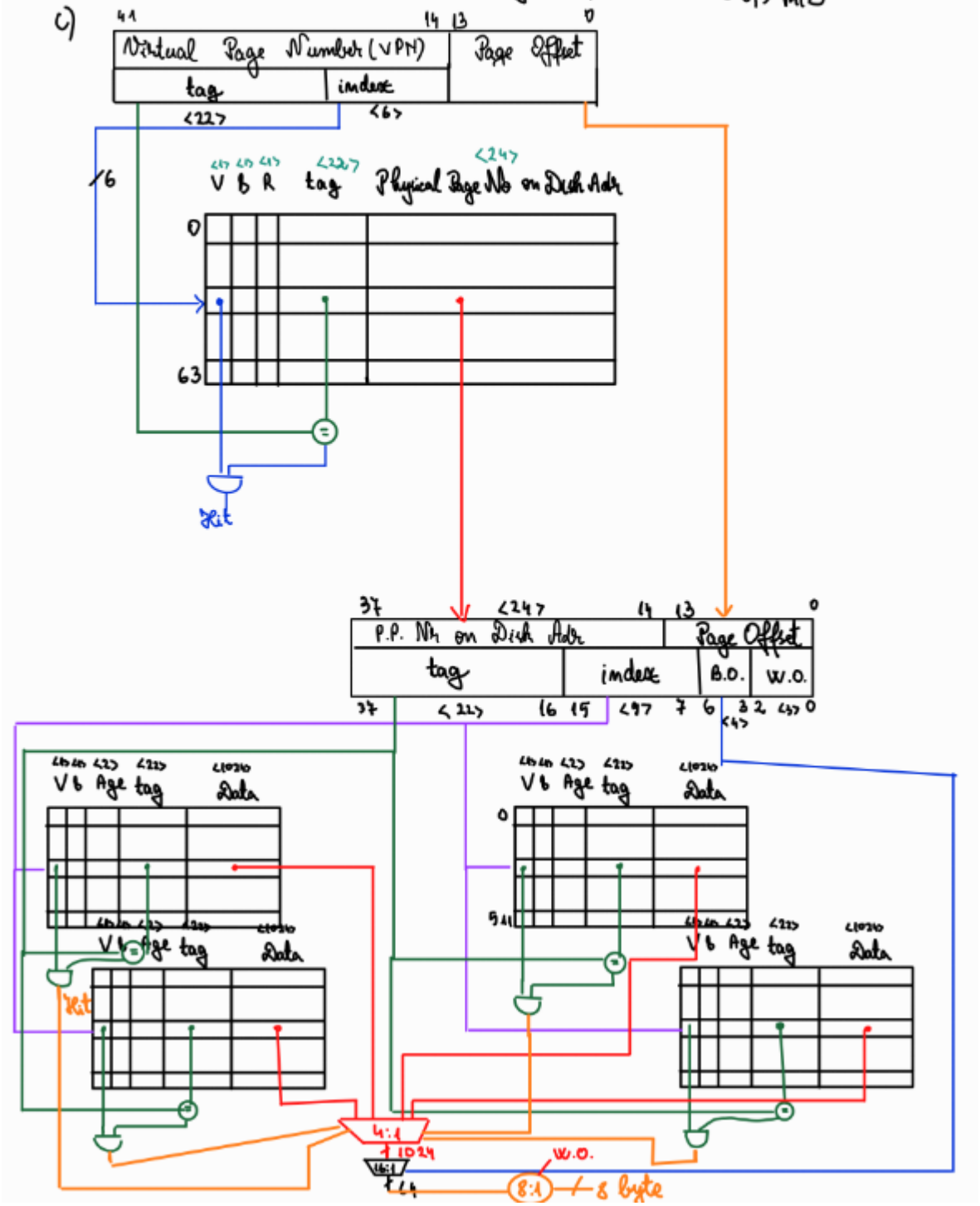
$tag_{TLB} = 42 - 14 - 6 = 22b$

$TLB_{\text{total size}} = 2^6 \left(\underbrace{22b}_{\text{tag}} + \underbrace{1b}_{V} + \underbrace{1b}_{\text{dirty}} + \underbrace{1b}_{\text{reference}} + 24B \right) = 2^6 \cdot 49b = 2^6 \cdot 2^3 \times 2^3 B$ (ind)

$\rightarrow 49b = 2^3$

$= 0,5\ KiB$

c)



Ex2 == Ex6

4) Virtual space = 64 TiB
Physical space = 128 GiB
byte-addressable
1 word = 64 bits = 8B
1 block = 8 words
1 page = 64 KiB
TLB [32 entries
full assoc.
Cache 8-way data = 512 KiB
WT + WA
2-way SA
- random replacement

4a) Address format
↳ virtual
↳ physical

4b) TLB, Cache total size

4c) Schema bloc comun

a) VA $64TiB = 2^6 \cdot 2^{40}B = 2^{46}B$
PA $128GiB = 2^7 \cdot 2^{30}B = 2^{37}B$
Page $64KiB = 2^6 \cdot 2^{10}B = 2^{16}B$

46 − 16 = 30 ; 37 − 16 = 21

Virtual address: [45 ... 16 | 15 ... 0] = [Virtual Page Number | Page offset] <30> <16>

1 cache block = 2^3 words
1 word = 2^3 B

Physical address: [36 ... 15 | 15 ... 0] = [Physical page no. or disk address | Page offset] <21> <16>

30 - block offset
WO - word offset

37 − 18 = 19

For cache: [36 ... 17 | 16 ... 6 | 5 4 | 3 2 1 0] = [Tag | Index | BO | WO] <19> <12> <2> <3>

1 cache block = 2^3 words = $2^3 \cdot 2^3 B = 2^6 B$/block

512 KiB data cache = $2^9 \cdot 2^{10} B = 2^{19} B$ ⇒ $\frac{2^{19} B}{2^1 s.a.} = 2^{18} B$/bank

$\frac{2^{18} B/\text{bank}}{2^6 B/\text{block}} = 2^{12}$ blocks/bank → Index size

b) Cache total size = $\underbrace{2}_{\text{s.a}} \times \underbrace{2^{12}}_{\text{blocks/bank}} \left(\underbrace{1\text{ bit}}_{V} + \underbrace{19\text{ bits}}_{\text{tag}} + \underbrace{2^6 B}_{\text{data}} \right) =$

$= 2^{19} B \text{ data} + 2^{12} \times 20 \text{ bits} \cong 512\,KiB + 3 \cdot 2^{12} B$

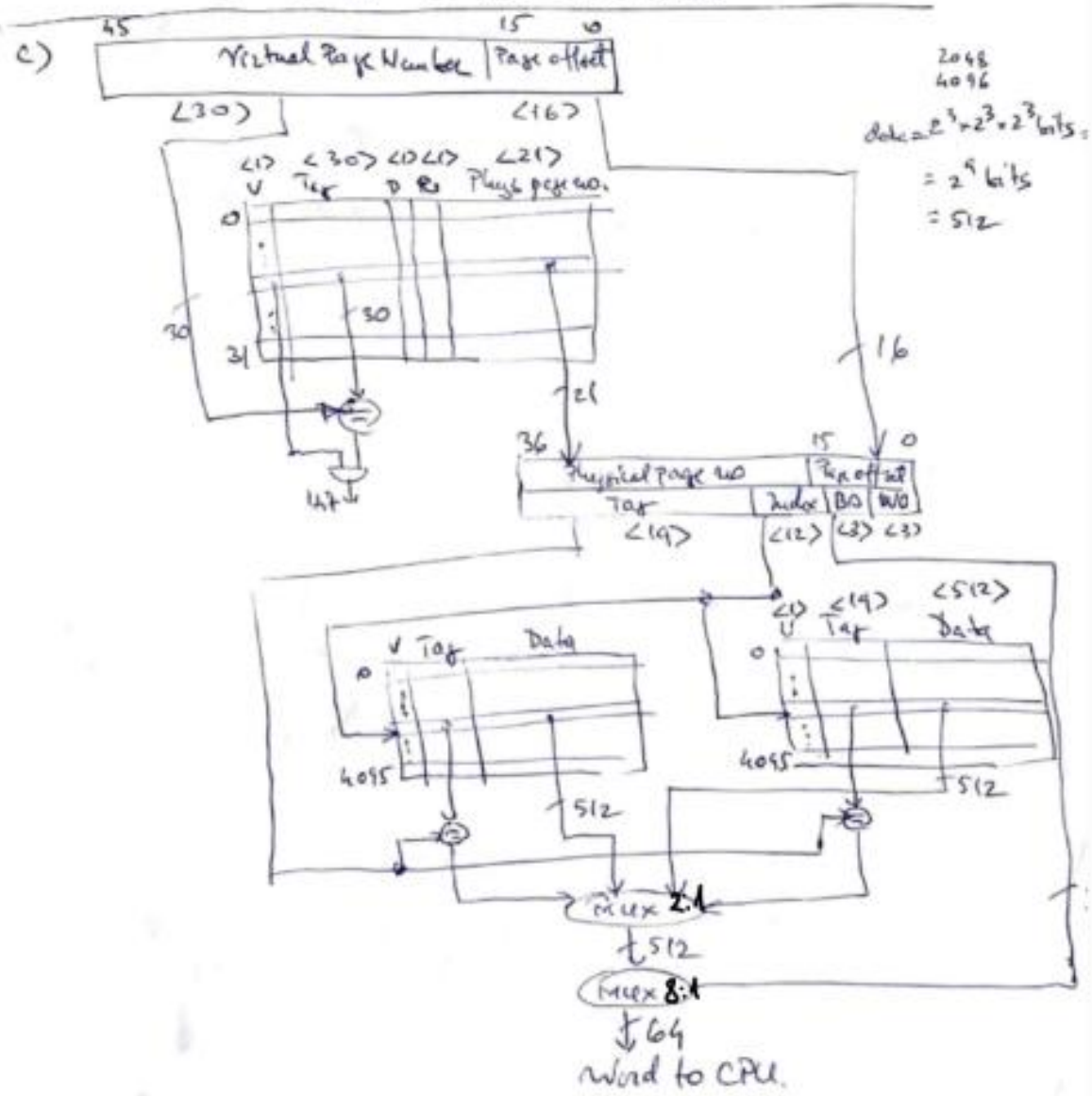
$= (512 + 12)\,KiB = 524\,KiB$

TLB total size = $2^5 \times (1\text{ bit valid} + 1\text{ bit dirty} + 1\text{ bit acc} +$

$+ 30\text{ bits tag} + 21\text{ bits phys. addr}) \cong 2^5 \times 64 \text{ bits} =$

$= 2^5 \times 2^3 B = 2^8 B = 0.25\,KiB$

c)



(4) Se dă un sistem de memorie, cu spațiul virtual de dimensiune I
(3p) 256 TiB, spațiu fizic de 256 GiB și adresare la nivel de octet. Cuvântul este pe 64 de biți, block-ul are 64 de cuvinte. O pagină de memorie are dimensiunea de 64 KiB, iar dimensiunea de date a cache-ului (SA pe 2 căi) este de 256 KiB. Cache-ul este Write Back, cu Write Allocate în caz de Write Miss și are o politică de înlocuire random. TLB-ul are 64 de intrări și folosește mapare directă.
Se cer:

(1p) a) Formatul adresei de memorie virtuală respectiv fizică
(1p) b) Dimensiunea totală a memoriei cache și a TLB-ului.
(1p) c) Schema bloc a mapării adresă fizică - adresă virtuală ce conține memoria cache și TLB-ul (ca în curs).

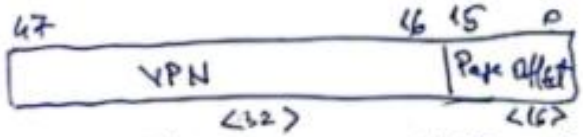
a) $VM_{Size} = 2^8 \times 2^{40} B = 2^{48} B$ $\quad Page = 64 KiB = 2^{16} B$

$PM_{Size} = 2^{38} B$

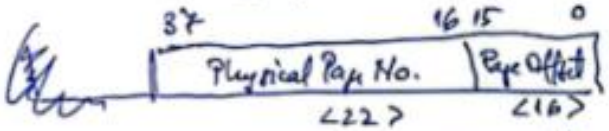
$TLB\ entries = 2^{E}$ → TLB Index

$1\ word = 2^3 B$ → Word Offset

$1\ block = 2^6\ words = 2^6 \cdot 2^3 B = 2^9 B$ → Block Offset



Virtual Memory

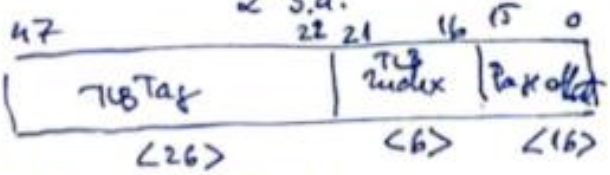


Physical Memory

38 − 16 = 22

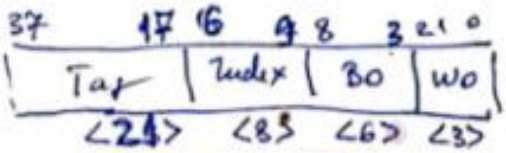
Cache data size $= 256 KiB = 2^{18} B = \frac{2^{18} B}{2^9 B/block} = 2^9$ blocks

Bank size $= \frac{2^9 \text{ blocks}}{2^1 \text{ S.a.}} = 2^{8}$ → Index blocks/bank.



Virt Mem

48 − 16 = 32 ; 32 + 1 = 33 ; 17 + 23 = 40



Phys. Mem

b) $CacheTSize = \underbrace{2^1}_{s.a} \times \underbrace{2^8}_{index} \times (\underbrace{2^9 B}_{data\,block} + \underbrace{16}_{V} + \underbrace{26}_{Tag} + \underbrace{16}_{dirty})$ $= 2^9 \times 2^9 B + 2^9 \cdot 3B =$

$= 2^{18}_{B} + 3 \times 0{,}5\,KiB$

$= 256\,KiB + 1{,}5\,KiB$

$= 257{,}5\,KiB$

$\cong 3B$

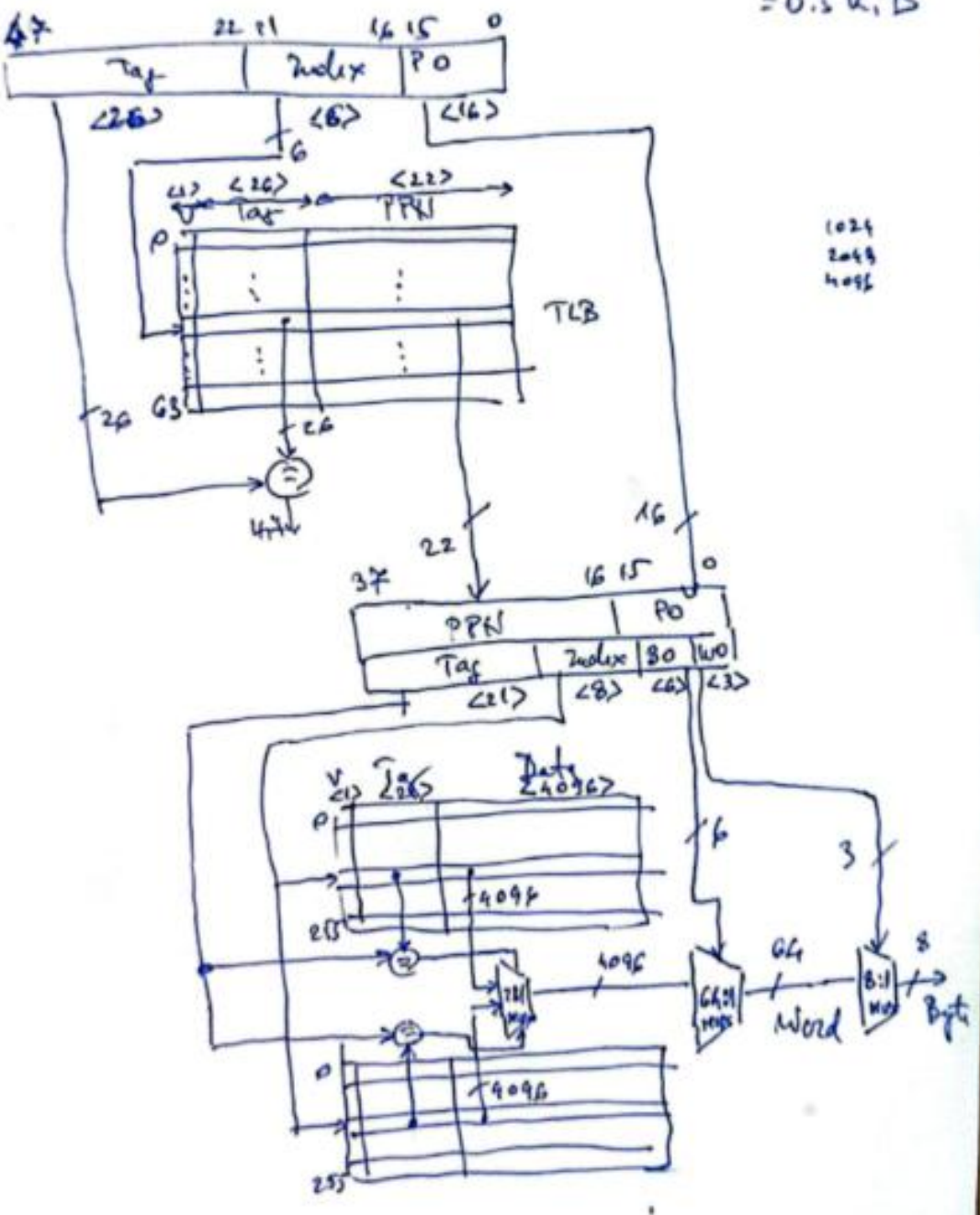
16 dirty

$TLBTSize = 2^6 \times (\underbrace{1}_{V} + \underbrace{26}_{TLBtag} + 22)\,bits = 2^6 \times 49b \cong 2^6 \times 2^3 B$

$\cong 2^3 B = 2^9 B$

$= 0{,}5\,KiB$

c)



Ex 4. (4) Se dă un sistem de memorie cu spațiul virtual de dimensiune 512 TiB, spațiu fizic de 128 GiB și adresare la nivel de octet. Cuvântul este pe 64 de biți, blocul are 32 de cuvinte. O pagină de memorie are dimensiunea de 16 KiB, iar dimensiunea de date a cache-ului (4 căi) este de 128 KiB. Cache-ul este Write Through cu Write No Allocate în caz de Write Miss și are o politică de înlocuire de tip LRU cu age register. TLB-ul are 128 de intrări și folosește mapare set-asociativă pe 2 căi. Se cer:

(1p) b) Dimensiunea totală a memoriei cache și a TLB-ului

(1p) a) Formatul adresei de memorie virtuală, respectiv fizică

(1p) c) Schema bloc a mapării adresă virtuală – adresă fizică ce conține memoria cache și TLB-ul (ca în curs și Fast MATH.)

a) Virtual Space → $512\,TiB = 2^{49}B$

Physical Space → $128\,GiB = 2^{37}B$

1 word = 64 biți = $2^3 B$

1 block = 2^5 words = $2^5 \cdot 2^3 B = 2^8 B$

1 page = $2^{14} B$

Cache data size = $2^{17} B$ = $\frac{2^{17} B}{2^8 B/block} = 2^9$ blocks

Bank size = $\frac{2^9 blocks}{2^{2\,s.a}} = 2^7$ blocks

Virtual address: 48 | Virtual Page Number <35> | Page offset <14> | 0

Physical address: 36 | Physical page no. <23> | Page offset <14> | 0

128 entries TLB = 2^7 entries

$\frac{2^7}{2^{1\,sa}} = 2^6$ indexes

Virtual address: 48 | TLB Tag <29> | Index TLB <6> | Block offset <14> | 0

Physical address: 36 | Tag <22> | Index <7> | Block offset <5> | Word offset <3> | 0

b) Cache total size = $\underbrace{2^2}_{s.a.} \times \underbrace{2^7}_{index} \times (\underbrace{2^8 B}_{data} + \underbrace{\frac{1}{V} + \frac{2}{age} + \underbrace{22}_{tag}}_{\cong 48}) =$

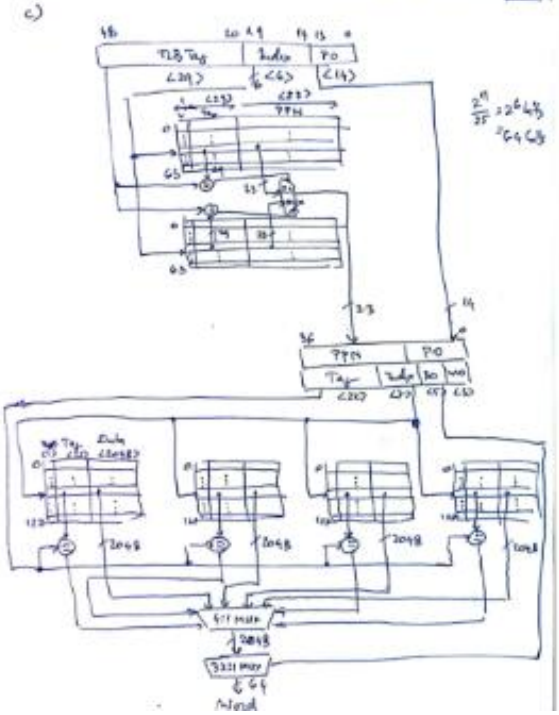
$= 130\,KiB$

$TLB_{size} = 2^7 \times (\frac{1}{V} + \frac{28}{tag} + \frac{23}{PPN}) \cong 2^7 (24+32)\,bits =$

$= 2^7 \cdot 7\,B \cong 1^7 \cdot 8\,B = 1\,KiB$

with CamScanner

c)



Ex 5

a) GVA: $2^{64}B = 2^{32}B$

GPA: $2^{32}B$

byte addressing

16 KiB page $\Rightarrow 2^4 2^{10} = 2^{14}B$

32-entry TLB (2-way SA)

256 KiB Data Cache (2-way SA) $= 2^{18}B = \frac{2^{18}}{2^3} = 2^{15}$ blocks

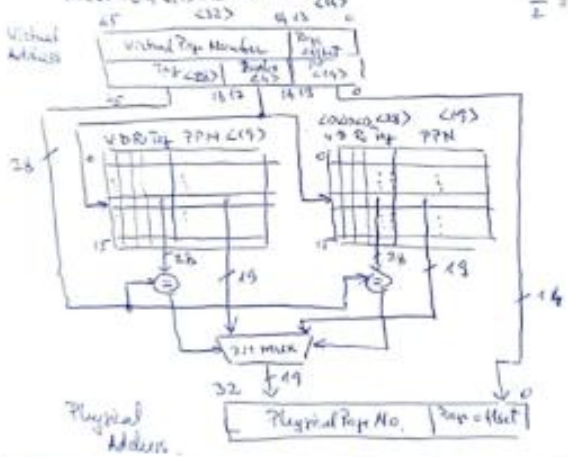
1 block = 16 words $= 2^4 w = 2^4 \cdot 2^3 B = 2^7 B$

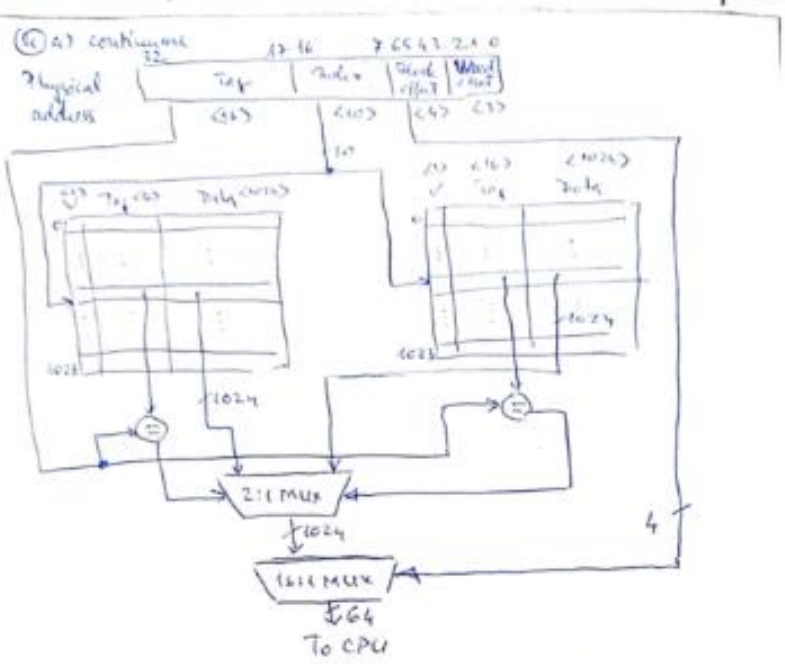
1 word = 64 bits $= 2^6$ bits $= 2^3 B$

$\frac{2^5}{2} = 2^4$ ④ index TLB

$\frac{2^{15}}{2} = 2^{14}$ blocks / bank

32 − 17 = 15





b) TLB Size = $\underbrace{2^5}_{\text{total entries}} \times (\underbrace{31 \text{ bits}}_{\text{Tag, V, D, R}} + \underbrace{19 \text{ bits}}_{\text{PPN}}) = 2^5 \times 50 \text{ bits}$

$\cong 2^5 \times 2^6 \text{ bits}$

$= 2^5 \times 2^3 B = 2^8 B$

$= 0.25 \text{ KiB}$

Cache Size total = $\underbrace{2}_{\text{S-a}} \times \underbrace{2^{10}}_{\text{i-bits}} \times (\underbrace{17 \text{ bits}}_{\text{CPN bits}} + 2^7 B)$

$= 2^{11} \times 17 \text{ bits} + 2^{18} B$

$\cong 2^{11} \times 2^5 \text{ bits} + \underbrace{256 \text{ KiB}}_{\text{data}} \cong 264 \text{ KiB}$

$2^{16} \text{ bits} = 2^{13} B = 8 \text{ KiB}$

Formule

1. WB + WA

Accesses via the BUS = freq $\left[\frac{\text{words}}{\text{sec}}\right]$ · MR · (% victim · WMP^{b}_{WB} + % victim · RMP^{b}_{WB})

Write Miss Penalty$^{b}_{WB}$ = Read Miss Penalty$^{b}_{WB}$

WMP^{b}_{WB} = $\frac{\text{block size}}{\text{BUS width}}$ · BUS Reads + $\frac{\text{block size}}{\text{BUS width}}$ · dirty · BUS writes

ALLOCATE UPDATE

2. WB + WNA

Accesses via the BUS = freq $\left[\frac{\text{words}}{\text{sec}}\right]$ · MR · (% victim · WMP^{b}_{WB} + % victim · RMP^{b}_{WB})

RMP^{b}_{WB} = $\frac{\text{block size}}{\text{BUS width}}$ · BUS Reads + $\frac{\text{block size}}{\text{BUS width}}$ · dirty · BUS writes

ALLOCATE UPDATE

WMP^{b}_{WB} = 1

3. WT + WA

Accesses via the BUS = freq $\left[\frac{\text{words}}{\text{sec}}\right]$ · MR · (% victim · WMP^{b}_{WT} + % victim · RMP^{b}_{WT}) +

pt. hit p. → + freq (1 − MR) · WHP^{b}_{WT} · % victim

RMP^{b}_{WT} = $\frac{\text{block size}}{\text{bus width}}$ BR

WMP^{b}_{WT} = $\frac{\text{block size}}{\text{bus width}}$ BR + 1 BW

WHP^{b}_{WT} = 1 BW

4. WT + WNA

Accesses via the BUS = freq $\left[\frac{\text{words}}{\text{sec}}\right]$ · MR · (% victim · WMP^{b}_{WT} + % victim · RMP^{b}_{WT}) +

pt. hit p. → + freq (1 − MR) · WHP^{b}_{WT} · % victim

RMP^{b}_{WT} = $\frac{\text{block size}}{\text{bus width}}$

WMP^{b}_{WT} = 1 BUS Write

WHP^{b}_{WT} = 1 BUS Write

Exc_1 Un sistem de calcul are un procesor ce trimite referințe către memoria cache la o frecvență de 10^8 words/second, unde 10% sunt scrieri, iar 90% citiri.

Cache-ul are un miss rate de 10% și un block de 8 cuvinte.

Bus-ul are un bandwidth de 10^9 acces/secundă și o lățime (bus bandwidth) de 1 cuvânt.

La orice moment, 20% din blocuri sunt dirty.

Să se calculeze procentajul din bus bandwidth folosit din cauza implementării cache-ului în următoarele cazuri: A_2, B_1 → date diferite

A1 a) WB + WA b) WT + WNA

a)

$$\text{Accesses via the BUS} = \text{freq}\left[\frac{\text{words}}{\text{sec}}\right] \cdot MR \cdot (\%\,\text{scrieri} \cdot WMP^{b}_{WB} + \%\,\text{citiri} \cdot RMP^{b}_{WB})$$

$$WMP^{b}_{WB} = \frac{\text{block size}}{\text{BUS width}} \cdot \text{BUS Reads} + \frac{\text{block size}}{\text{BUS width}} \cdot \text{dirty}^{WB} \cdot \text{BUS writes} =$$

ALLOCATE UPDATE

$$= \frac{8}{1} + 0{,}2 \cdot \frac{8}{1} = 1{,}2 \cdot 8 = 9{,}6 \text{ BUS Accesses}$$

$$AVTB = 10^8 \cdot 0{,}1 \cdot (0{,}1 \cdot 9{,}6 + 0{,}9 \cdot 9{,}6) = 10^7 \cdot 9{,}6 \text{ BUS Accesses}$$

$$\%\,\text{Used BUS}_{WB+WA} = \frac{AVTB}{\text{BUS Bandwidth}} = \frac{10^7 \cdot 9{,}6}{10^9} = 9{,}6\%$$

b)

$$\text{Accesses via the BUS} = \text{freq}\left[\frac{\text{words}}{\text{sec}}\right] \cdot MR \cdot (\%\,\text{scrieri} \cdot WMP^{b}_{WT} + \%\,\text{citiri} \cdot RMP^{b}_{WT}) +$$

pt. Hit P. → $+ \text{freq} \cdot (1 - MR) \cdot WHP^{b}_{WT} \cdot \%\,\text{scrieri}$

$$RMP^{b}_{WT} = \frac{\text{block size}}{\text{bus width}} = \frac{8}{1} = 8 \text{ BUS Reads}$$

$$WMP^{b}_{WT} = 1 \text{ BUS Write}$$

WMP^{B}_{WT} = 1 BUS Write

$AVTB = 10^8 \cdot 0,1 \cdot (0,1 \cdot 1 + 0,9 \cdot 8) + 10^8 \cdot 0,9 \cdot 1 \cdot 0,1 =$

$= 10^7 \cdot (7,3 + 0,9) = 8,2 \cdot 10^7$

$\%$ Used BUS $_{WT + WNA} = \frac{8,2 \cdot 10^7}{10^8} = 8,2\%$

B_1 am schimbat : a) WB + WNA b) WT + WA

a)

Accesses via the BUS = freq $\left[\frac{\text{words}}{\text{sec}}\right]$ · MR · ($\%$ schieb · WMP^{B}_{WB} + $\%$ citiri · RMP^{B}_{WB})

$RMP^{B}_{WB} = \frac{\text{block size}}{\text{BUS width}}$ · dirty BUS Writes =

WMP_{WB} = 1

①
```
for( i=0, p=0, m=0; i<mase; i++) {
    if( i%2==0){
        p++;
    }
    else  m++;
}
```

$r_0 \longrightarrow$ adr
$r_1 \longrightarrow$ i
$r_2 \longrightarrow$ p
$r_3 \longrightarrow$ m
$r_4 \longrightarrow$ mase

```
init:   MOV   r1 , #0
        MOV   r2 , # 0
        MOV   r3 , # 0
        AdR   r0 , mase
        LdR   r4 , [r0]

loop:   CMP  r1, r4      i - mase
        BGE  end
        ANd  r5 , r1 , #1
        CMP  r5, #0
        AddEQ  r2 , r2 , #1  ->p++
        AddNE  r3, r3, #1
        Add  r1, r1, #1
    B loop

end:  AdR  r0, p          AdR  r0, m
      STR  r2 , [r0]      STR  r3 , [r0]
```

② for (i=0, m=0 ; i < max; i++) {
    m = m+i;
    if(m > 2*max) m = 0;
}

r_0 → adr
r_1 → i
r_2 → m
r_3 → max

```
init: MOV  r1, #0
      MOV  r2, #0
      ABR  r0, max
      LBR  r3, [r0]
loop: CMP  r1, r3
      BGE  end
      Abb  r2, r2, r1
      Abb  r4, r3, r3
      CMP  r2, r4
      MOVGT r2, #0
      Abb  r1, r1, #1

  B loop
end:  AbR  r0, m
      STR  r2, [r0]
```

③ for(i=1, s=0 ; i<=m ; i++) {
    if (i %3 == 0) s += i*i ;
}

r_0 ⟶ adr
r_1 ⟶ i
r_2 ⟶ s
r_3 ⟶ m
r_4 ⟶ temp
r_5 ⟶ rest

```
init:  MOV  r1, #1
       MOV  r2, #0
       MOV  r0, m
       MOV  r3, [r0]
       MOV  r5, #1
loop:  CMP  r1, r3
       BGT  end
       CMP  r5, #0
       BNE  skip
       MUL  r4, r1, r1
       ADD  r2, r2, r4      s += i*i
skip:  ADD  r1, r1, #1
       ADD  r5, r5, #1
       CMP  r5, #3
       MOVEQ r5, #0
       B loop
end:   ADR  r0, s
       STR  r2, [r0]
```

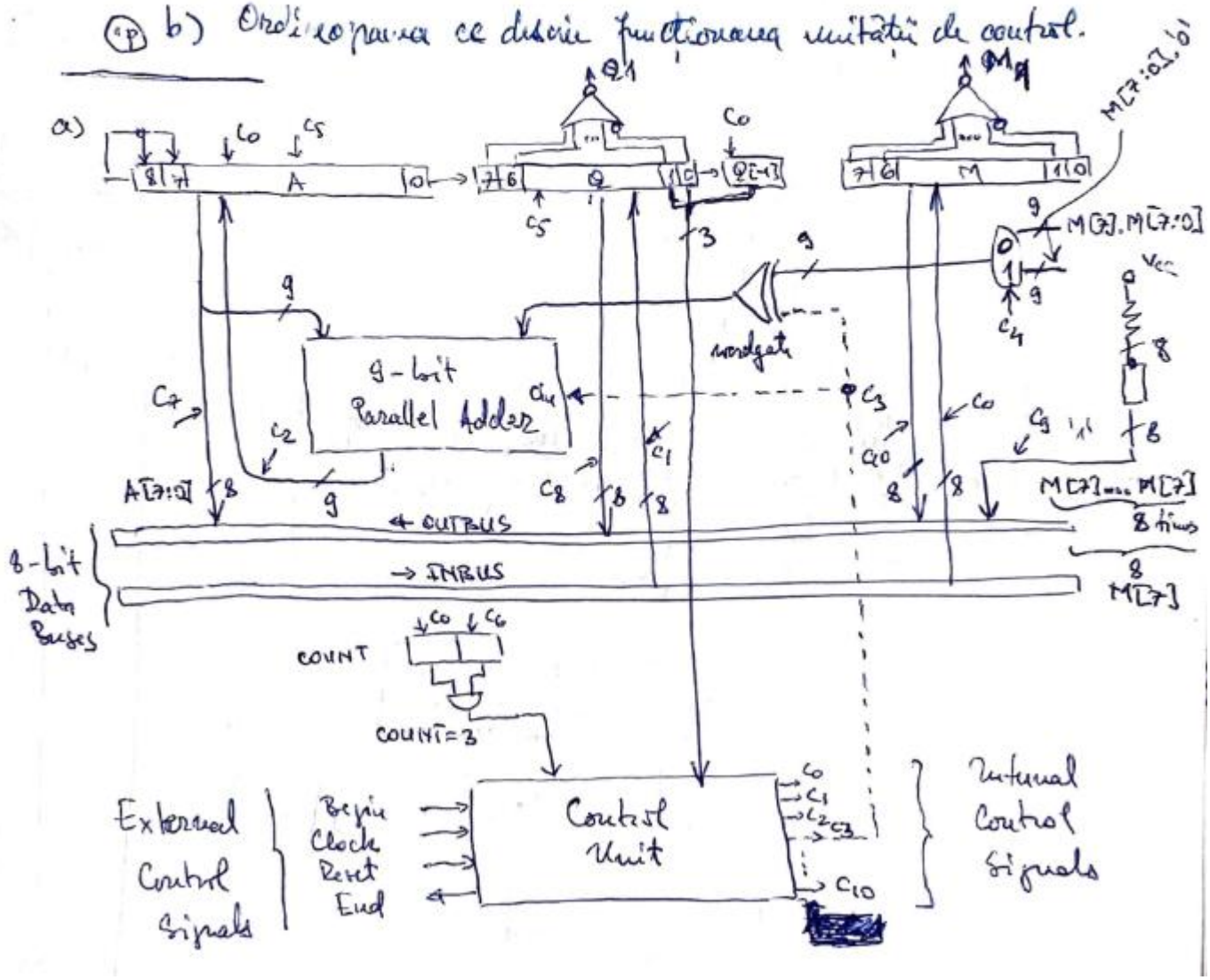
41
B2

(2p) (1) Să se completeze descrierea unui înmulțitor Booth radix 4, astfel încât să trateze cazul special al înmulțirii cu 1 (unul dintre operanzi este egal cu 1). Se cer:

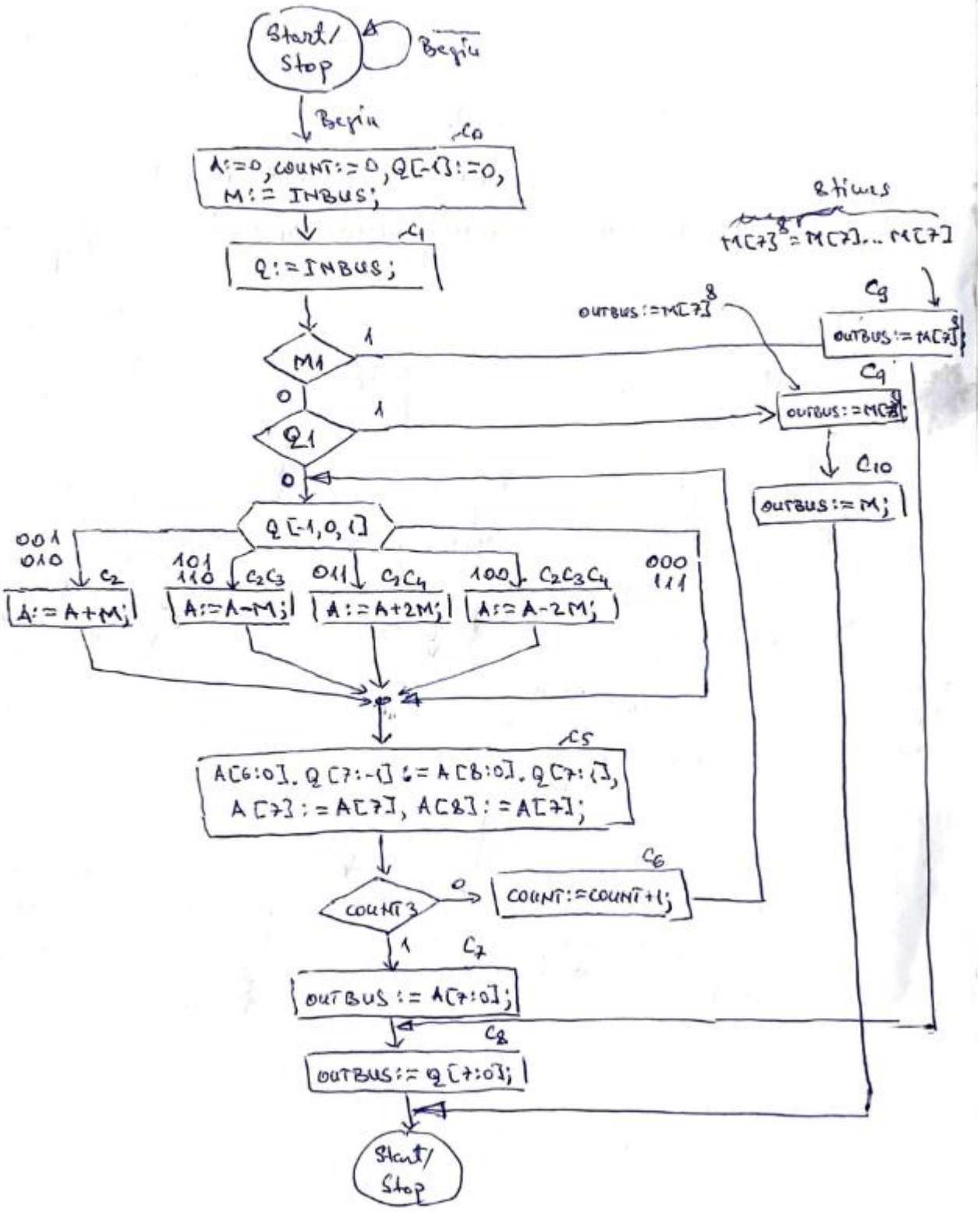
(1p) a) Schema bloc hardware + semnalele de control

(1p) b) Ordinograma ce descrie funcționarea unității de control.

a)



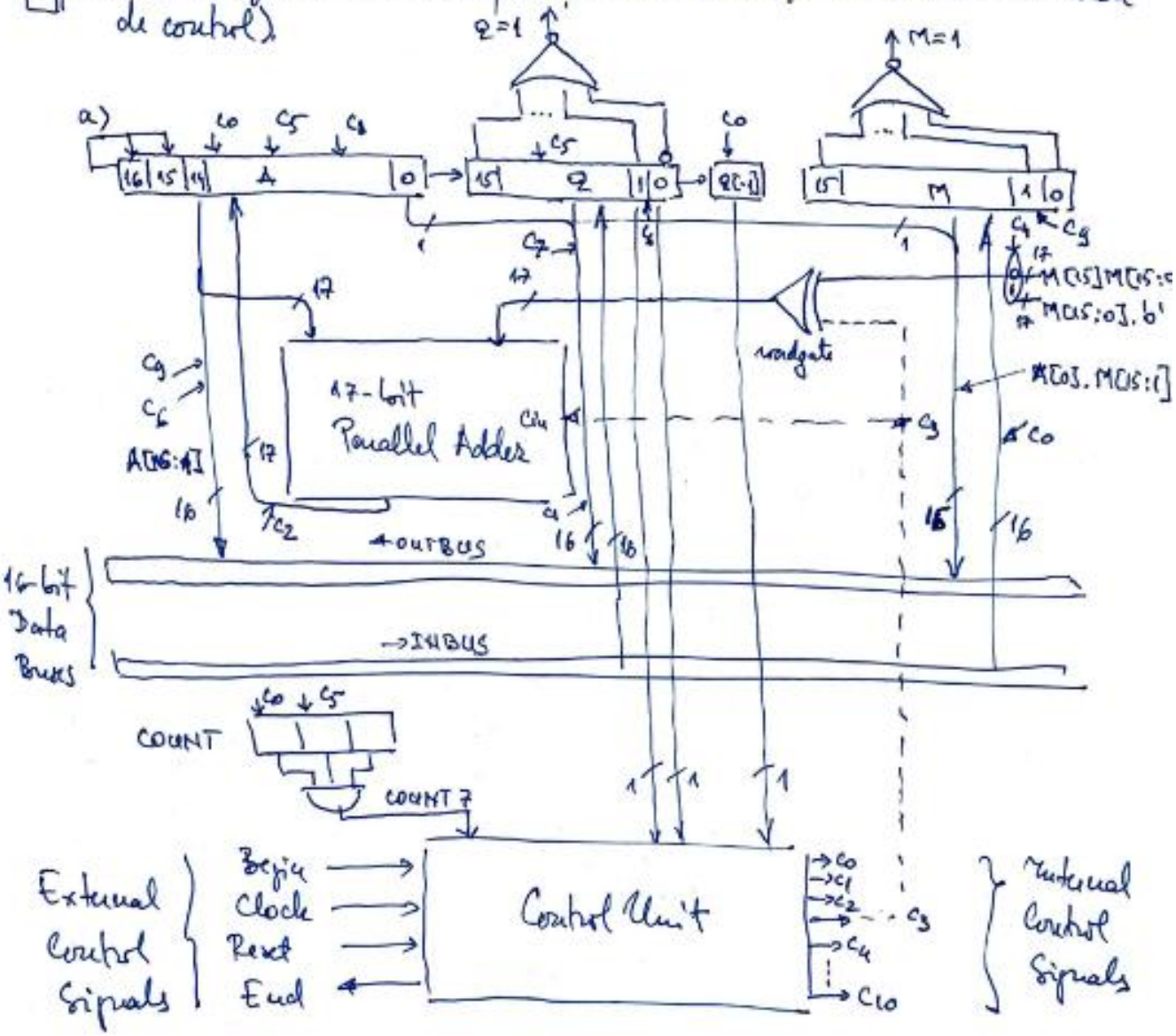
b)

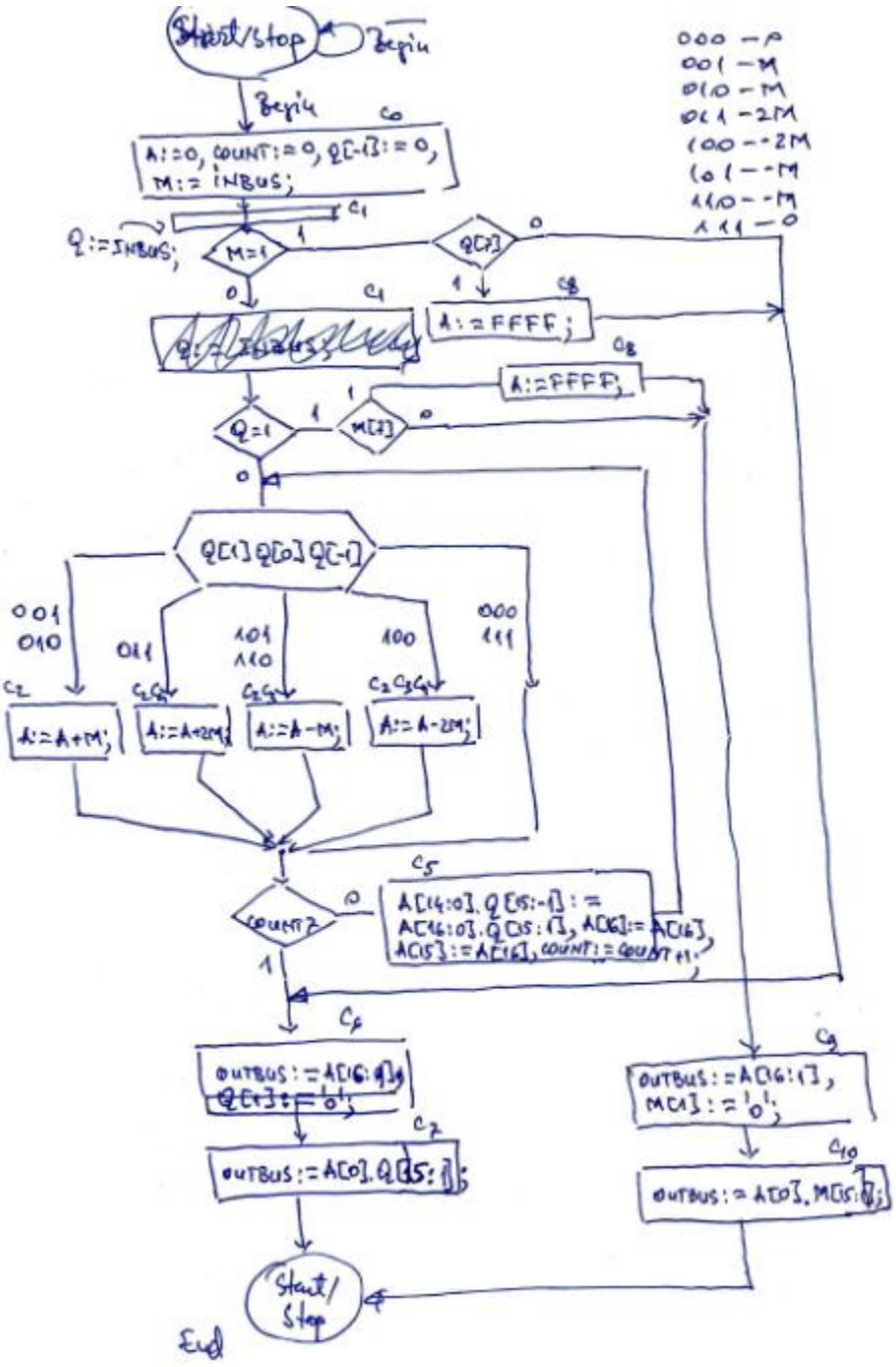

Start/ Stop
Begin
Begin
C0
A:=0, COUNT:=0, Q[-1]:=0, M:=INBUS;
C1
Q:=INBUS;
M1
1
0
Q1
1
0
Q[-1,0,1]
001
010
C2
A:=A+M;
101
110
C2C3
A:=A-M;
011
C1C4
A:=A+2M;
100
C2C3C4
A:=A-2M;
000
111
C5
A[6:0], Q[7:-1]:=A[8:0], Q[7:1],
A[7]:=A[7], A[8]:=A[7];
C6
COUNT<7
0
COUNT:=COUNT+1;
1
C7
OUTBUS:=A[7:0];
C8
OUTBUS:=Q[7:0];
Start/ Stop
8 times
M[7]=M[7]...M[7]
C9
OUTBUS:=M[7];
OUTBUS:=M[7]
C9
OUTBUS:=M[7];
C10
OUTBUS:=M;

(2p) ① Să se modifice dispozitivul de înmulțire Booth radix-4, astfel încât acesta să înmulțească numere fracționare reprez. pe 16 biți și să trateze în mod distinct situațiile de înmulțire cu unu. (Rezultatul este egal cu celălalt operand, exprimat pe 32 biți.) Se cere:

(1p) a) Schema bloc detaliată pe platforma ~~Neumann~~ Hayes. (cu semnale de ctrl.)

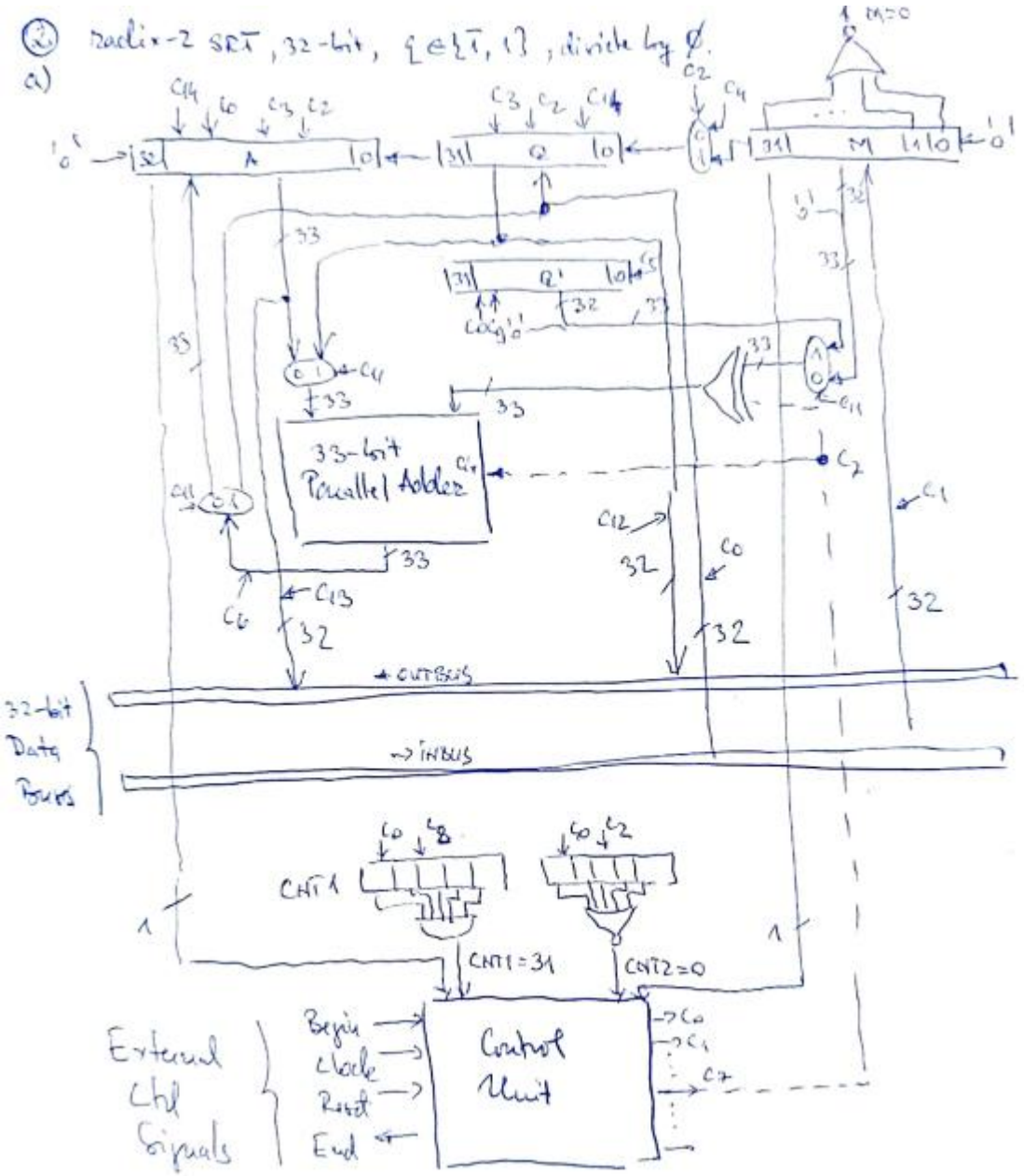
(1p) b) Ordinograma ce descrie funcționarea unității de control (cu semnalele de control).



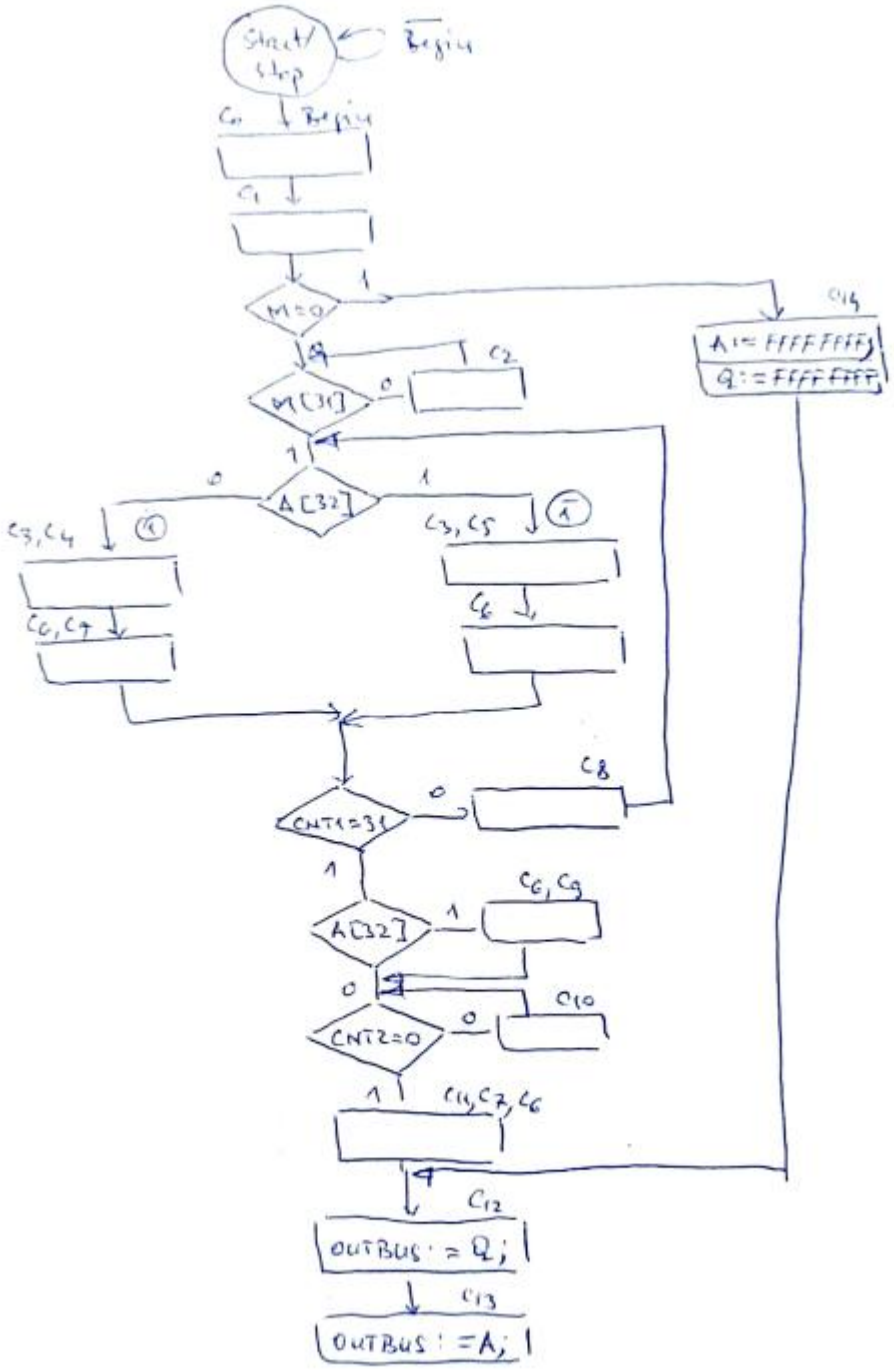

Start/Stop
Begin
Begin
c0
A:=0, COUNT:=0, Q[-1]:=0,
M:=INBUS;
c1
Q:=INBUS;
M=1
1
0
Q[7]
0
1
c4
c8
A:=FFFF;
A:=FFFF;
Q=1
1
M[7]
1
0
0
Q[1] Q[0] Q[-1]
001
010
011
101
110
100
000
111
c2
c2c4
c2c3
c2c3c4
A:=A+M;
A:=A+2M;
A:=A-M;
A:=A-2M;
COUNT
0
c5
A[14:0], Q[15:-1] :=
A[16:0], Q[15:1], A[16]:=A[16],
A[15]:=A[16], COUNT:=COUNT+1;
1
c6
OUTBUS:=A[16:1];
Q[15]:=!0!;
c7
OUTBUS:=A[0], Q[15:1];
c9
OUTBUS:=A[16:1],
M[1]:=!0!;
c10
OUTBUS:=A[0], M[15:1];
Start/
Stop
End
000 - P
001 - M
010 - M
011 - 2M
100 - -2M
101 - -M
110 - -M
111 - 0

(2) radix-2 SRT, 32-bit, $q \in \{-1, 1\}$, divide by 0.

a)



2.5)


Start/ Stop
Begin
C0
C1
M=0
C14
A:= FFFFFFFF
Q:= FFFFFFFF
C2
M[31]
A[32]
C3, C4
C5, C6, C7
C3, C5
C6
C8
CNT1=31
C6, C9
A[32]
C10
CNT2=0
C11, C7, C6
C12
OUTBUS := Q;
C13
OUTBUS := A;

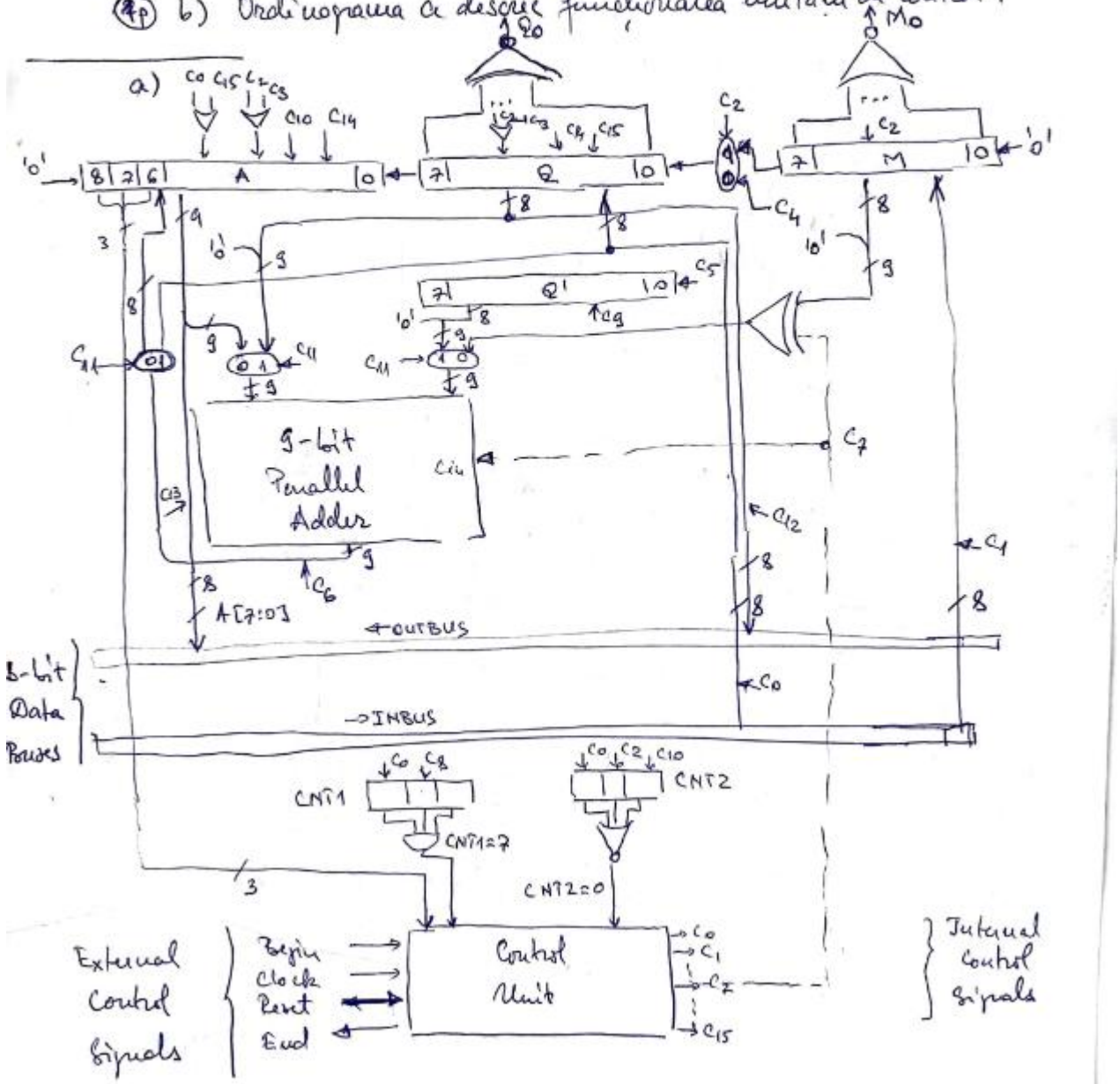
A2
B1

2p (A) Să se completeze descrierea unui împărțitor SRT radix 2, astfel încât să se trateze cazurile speciale de împărțire la zero (când atât câtul cât și restul sunt "1...1") și împărțirea a lui zero. Se cer.

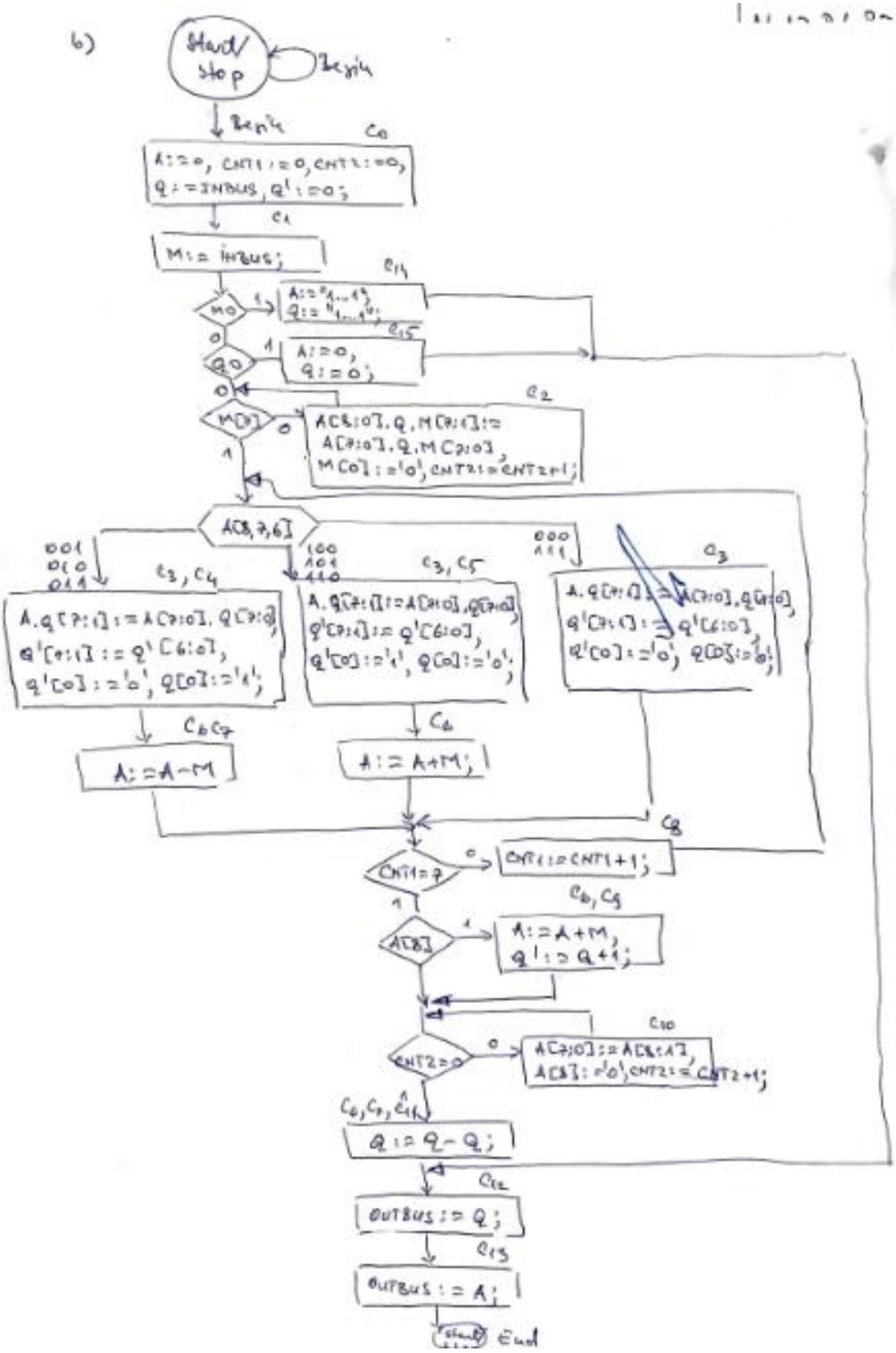
(1p) a) Schema bloc hardware + semnale de control.

(4p) b) Ordinograma ce descrie funcționarea unității de control.

a)



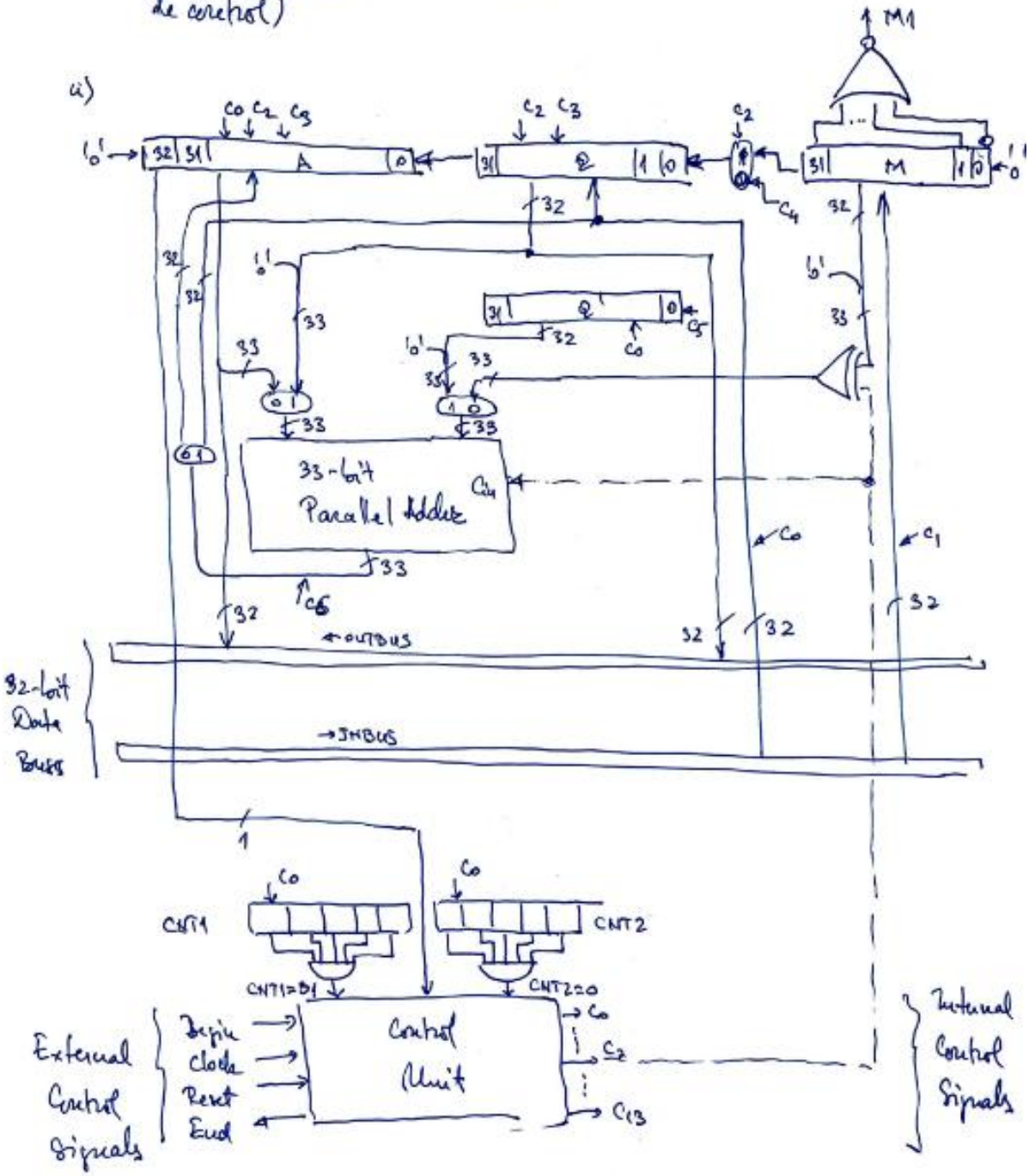
b)


Start/ stop
Reset
Reset
c0
A:=0, CNT1:=0, CNT2:=0;
Q:=INBUS, Q':=0;
c1
M:= INBUS;
c14
M0
1
A:="1...1";
Q:="1...1";
c15
0
Q0
1
A:=0;
Q:=0;
0
M[0]
0
c2
ACC[8:0].Q, M[7:1]:=
A[7:0].Q, M[2:0],
M[0]:='0', CNT2:=CNT2+1;
1
ACC[8,7,6]
001
010
011
c3, c4
A.Q[7:1]:=A[7:0].Q[7:0],
Q'[7:1]:=Q'[6:0],
Q'[0]:='0', Q[0]:='1';
100
101
110
c3, c5
A.Q[7:1]:=A[7:0],Q[7:0],
Q'[7:1]:=Q'[6:0],
Q'[0]:='1', Q[0]:='0';
000
111
c3
A.Q[7:1]:=A[7:0],Q[0:0],
Q'[7:1]:=Q'[6:0],
Q'[0]:='0', Q[0]:='0';
C6C7
A:=A-M
C6
A:=A+M;
C8
CNT1=7
0
CNT1:=CNT1+1;
1
C6, C9
ACC8
1
A:=A+M,
Q':=Q+1;
C10
CNT2=0
0
A[7:0]:=ACC[7:0],
ACC8]:='0', CNT2:=CNT2+1;
C6, C9, C11
Q:=Q-Q;
C12
OUTBUS:=Q;
C13
OUTBUS:=A;
End

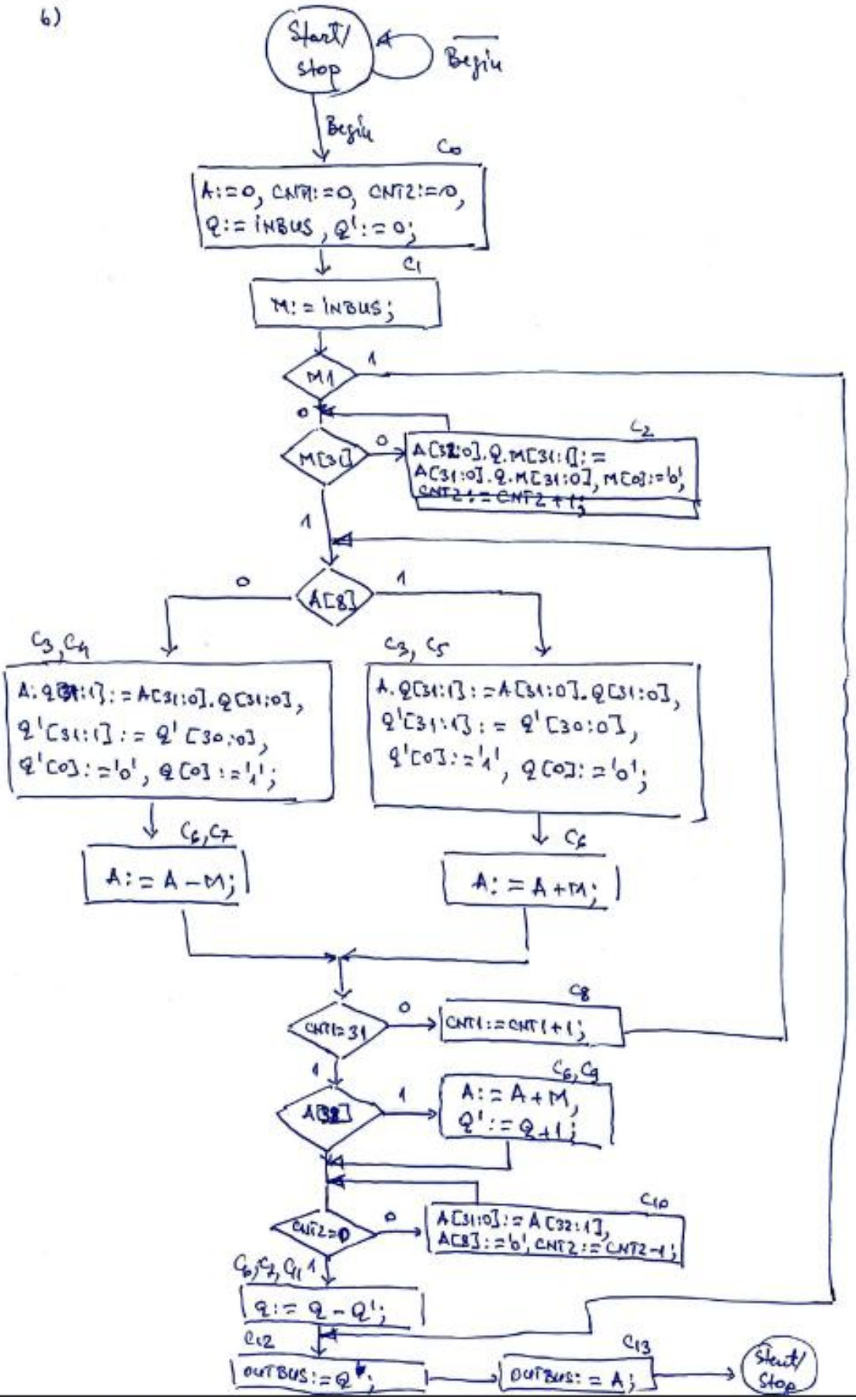
① Să se modifice dispozitivul de împărțire SRT radix-2 astfel încât acesta
(2p) să împartă numere pe 32 de biți fără a genera cifra câtului 0 (doar 1 și $\bar{1}$) și să trateze în mod distinct situația de împărțire la 1, de care:

a) Schema bloc hardware folosind platforma Hayes + semnalele de control

b) Ordinograma ce descrie funcționarea unității de control (+ semnalele de control)



6)


Start/ stop
Begin
Begin
C0
A:=0, CNT1:=0, CNT2:=0,
Q:= INBUS, Q':=0;
C1
M:= INBUS;
M1
1
0
M[31]
0
C2
A[32:0].Q.M[31:1]:=
A[31:0].Q.M[31:0], M[0]:='0';
CNT2:=CNT2+1;
1
A[31]
0
1
C3, C4
A.Q[31:1]:=A[31:0].Q[30:0],
Q'[31:1]:=Q'[30:0],
Q'[0]:='0', Q[0]:='1';
C3, C5
A.Q[31:1]:=A[31:0].Q[30:0],
Q'[31:1]:=Q'[30:0],
Q'[0]:='1', Q[0]:='0';
C6, C7
A:=A-M;
C6
A:=A+M;
CNT1=31
0
C8
CNT1:=CNT1+1;
1
A[32]
1
C6, C9
A:=A+M,
Q':=Q+1;
CNT2=0
0
C10
A[31:0]:=A[32:1],
A[31]:='0', CNT2:=CNT2-1;
C6, C7, C11
1
Q:=Q-Q';
C12
OUTBUS:=Q';
C13
OUTBUS:=A;
Start/ Stop